

Technical Exercise

Internal Support Desk — a basic corporate ticketing system

Prepared for Ibrahim Qureshi · Junior Full-Stack Developer

Deadline: Monday 6 July 2026. Build a basic working web app, then come to your second interview ready to demo it live and walk us through how it works.

Thanks for a great first conversation. This short task is a chance to show how you design and ship a small full-stack app end to end. It's deliberately kept basic — we care far more about clear thinking, sound fundamentals, and how you explain your decisions than about how many features you fit in. Please don't overwork it.

1 What to build

A simple internal support desk: a web app where staff can raise support tickets and a support team manages them through to resolution — think of a small company's internal IT help desk. There are two kinds of user:

- **Requester** — raises and tracks their own tickets.
- **Support agent** — sees the ticket queue, picks up tickets, and works them to resolution.

Core requirements (must-haves)

1. **Authentication** — users can register and sign in (use Supabase Auth).
2. **Roles & permissions** — at least two roles (Requester and Support Agent) with clearly different access.
3. **Raise a ticket** — title, description, category, and priority; new tickets start as "Open".
4. **Ticket queue** — a list view with status badges. Requesters see only their own tickets; agents see the full queue. Include at least filtering by status and sorting.
5. **Ticket detail & workflow** — open a ticket, move it through a defined flow (Open → In Progress → Resolved → Closed), and assign it to an agent.
6. **Comments / activity** — allow replies or comments on a ticket, shown as a timeline.
7. **Enforced access control** — a requester must not be able to see or change anyone else's tickets. Enforce this on the server / with Supabase Row-Level Security — not just by hiding buttons in the UI.
8. **A simple dashboard** — a basic overview for agents (e.g. ticket counts by status).

Optional stretch (only if you have time — not required)

- An Admin role with basic user management
- Search across tickets, or pagination on the queue
- File attachments on tickets (Supabase Storage)
- An audit log of status changes and assignments
- An in-app or email notification when a ticket is assigned or changes status

2 Design & look and feel

It should look and feel like a professional internal tool, not a prototype:

- A consistent UI built with a component library (e.g. shadcn/ui + Tailwind — your choice).
- A clear app layout: persistent sidebar navigation and a top bar.
- A proper data table: status badges, sortable columns, and filters.
- Real loading, empty, and error states — not blank screens.
- Inline form validation with clear, helpful error messages.
- Role-aware UI, so people only see what's relevant to them.
- A restrained, corporate palette with clean spacing and typography. Responsive is a plus; desktop is the priority.

3 Tech stack & hosting

- **App & server:** Next.js (App Router) is our suggestion, with server-side logic in route handlers or server actions.
- **Data, auth & security:** Supabase (Postgres, Auth, Row-Level Security), free tier.
- **Hosting:** deploy to a public URL on Vercel (free Hobby tier), under your own personal account.
- Everything must run on free tiers — no paid services required.

Heads-up: Supabase pauses free projects after about 7 days of inactivity. Make sure your project is awake when you demo — open the app shortly before the call.

4 Ground rules

Build it from scratch. You may not use, clone, fork, or adapt an existing open-source ticketing or help-desk system — such as (but not limited to) osTicket, Zammad, FreeScout, UVdesk, or Chatwoot. The ticketing application, its data model, and its logic must be your own work.

What's absolutely fine: general-purpose frameworks and libraries (Next.js, React, the Supabase client, a UI component library, form/validation libraries), a standard starter such as create-next-app, and AI coding assistants like Cursor or Claude Code. We know you use them — that's expected. The live walkthrough is how we'll explore your understanding.

5 Architecture documentation (required)

Alongside the app, write clear documentation explaining how it works — to a professional, client-facing standard, as if you were handing it to a client or a new engineer joining the project. It should cover:

- The stack you chose, and why.
- The data model — your tables and how they relate (a simple diagram is welcome).
- How authentication and authorization work, including your roles and Row-Level Security approach.
- The ticket workflow / status model.
- Key decisions and trade-offs, and what you'd do with more time.
- How to run the project locally.

A well-organised README in the repo is fine, or a separate document — your call, as long as it's clear and professional.

6 What to hand in

By **Monday 6 July 2026**, please share:

1. A link to the deployed, working app (Vercel).
2. A link to the source code (GitHub).
3. Your architecture documentation.

Then come to your second interview ready to demo the app live and talk us through the code and the decisions behind it.

7 How we'll assess it

We're looking at: a working core flow; a sensible data model; access control genuinely enforced on the server; clean, organised, readable code; a professional-looking UI; clear documentation; and — most of all — how well you explain your reasoning. For a task like this, sound fundamentals and good judgement matter far more than feature count or polish on every edge.

Keep it basic and time-boxed — a focused effort over the next few days, not your whole week. If anything's unclear or you hit a blocker, just ask; how you handle questions is part of the picture too. Good luck — we're looking forward to seeing what you build.